

LECTURE 01

Linux

History & Architecture

What an OS does, why Linux was born, and how it's put together — from the kernel up.

```
$ whoami
linus@kernel
$ cat os.txt
Unix-like, open-source,
monolithic kernel.
$ uname -a
Linux 1991 -> today
$ _
```

Tux says hi

Learning Objectives

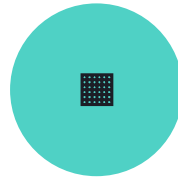
By the end of this lecture, you should be able to:



01

Explain the OS / kernel

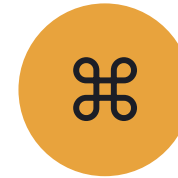
What an operating system and kernel actually do, and why Linux was created as a free Unix alternative.



02

Describe the architecture

The layered structure of Linux — from hardware up through the kernel, libraries, shell, and applications.



03

Differentiate distributions

How distros package the kernel with GNU tools, package managers, and desktop environments differently.

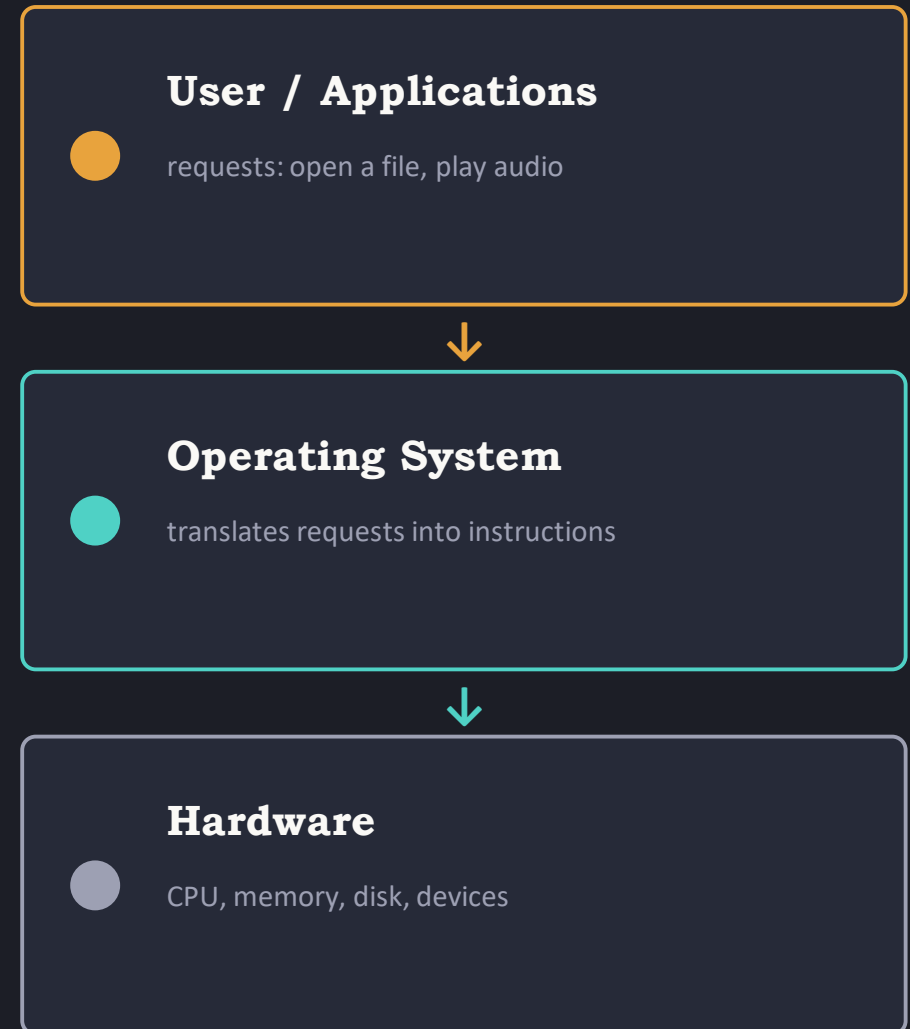
Why Do We Need an OS?

An operating system sits between the user/applications and the hardware, **translating high-level requests** (open a file, play audio) into instructions the CPU and devices understand.

Without it, every program would need to manage hardware directly.

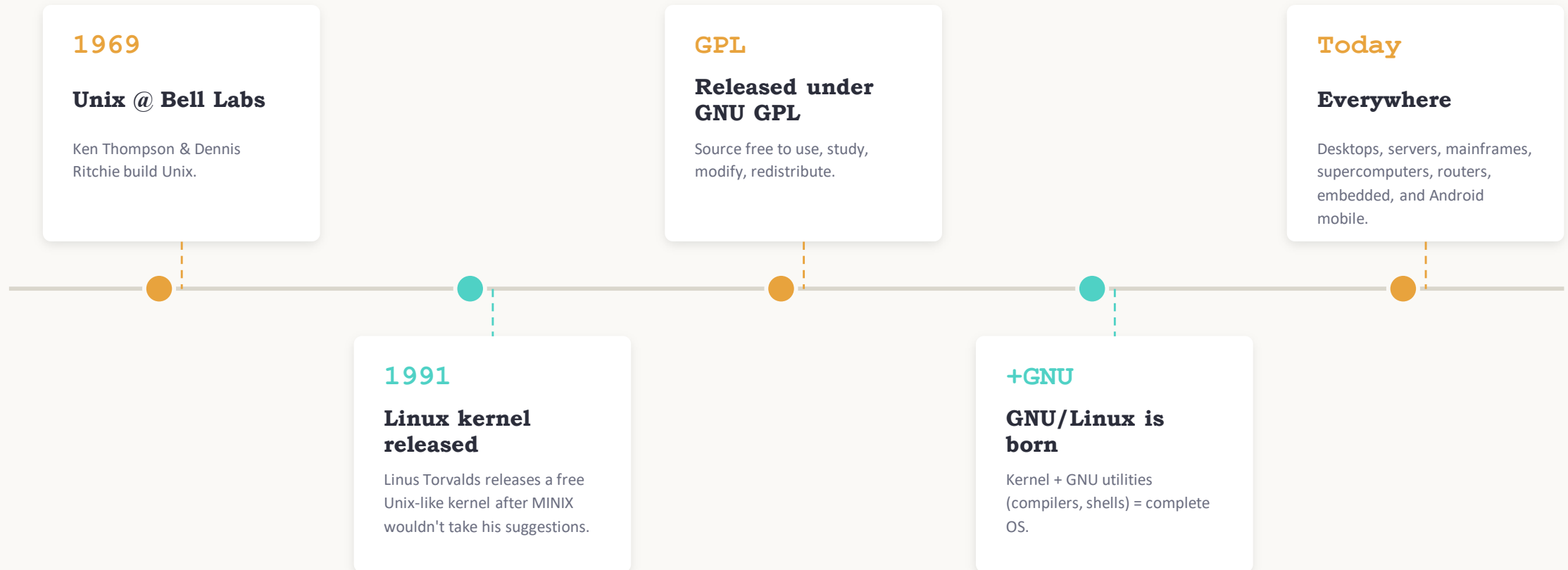
Linux, defined

An open-source, Unix-like operating system built around the Linux kernel.



A Short History

From Bell Labs to a Finnish CS student's side project



The GNU General Public License

Linux was released under the GPL — making its source code **free** in four concrete ways:



Use

Run the software for any purpose.



Study

Read and understand the source code.



Modify

Change it to fit your needs.



Redistribute

Share it — original or modified.

Kernel + GNU utilities (compilers, shells) = a complete OS — commonly called

GNU/Linux

Linux Distributions

A distribution (distro) = **kernel + GNU utilities + package manager + desktop environment**, packaged together.



Ubuntu

Beginner-friendly desktop



Debian

Stable, community-run



Fedora

Cutting-edge, Red Hat-backed



CentOS / RHEL

Enterprise servers



Linux Mint

Familiar desktop feel



Arch

Minimal, roll-your-own



Kali

Security & pen-testing

600+

distributions for desktop, server, security & embedded

The Layered Architecture

From hardware at the bottom to applications at the top



Applications

ls, cp, editors, compilers, user programs



Shell

command interpreter — bash, sh, csh, ksh



System Libraries

glibc — functions apps use to make system calls



Linux Kernel

process mgmt • memory mgmt • file system • device drivers • network stack



Hardware

CPU, memory, disk, network interfaces

Interface roles

Kernel

Manages CPU scheduling, memory, processes & hardware. Runs in privileged mode.

Shell

Takes user commands, passes them to the kernel — the user↔kernel interface.

System libraries

Provide functions apps use to make system calls (e.g. glibc).

Key Point for Exams

Linux is monolithic — be ready to draw the diagram and explain why.

MONOLITHIC — Linux

Process mgmt

Memory mgmt

File system

Device drivers

Network stack

all one large program, single address space

Runs in kernel space for speed: no message-passing overhead between subsystems.

MICROKERNEL — for contrast

Process mgmt

Memory mgmt

File system

Device drivers

Network stack

separate services, communicate via message passing

More fault-isolated, but slower due to inter-process communication overhead.

Recap Questions

Check yourself before moving on

01 Who created Linux, and in what year?

02 What license is Linux released under, and what does that guarantee users?

03 Draw and label the Linux architecture.